

УДК 519.614

О БИЛИНЕЙНОЙ СЛОЖНОСТИ И ПРАКТИЧЕСКИХ АЛГОРИТМАХ УМНОЖЕНИЯ МАТРИЦ

© 2013 г. А. В. Смирнов

(109028 Москва, Хохловский пер., 13, стр. 2., ФБУ РФЦСЭ при Минюсте России)

e-mail: S.Alexey.V@rambler.ru

Поступила в редакцию 19.03.2013 г.

Переработанный вариант 04.06.2013 г.

Предложен метод расчета билинейных алгоритмов умножения матриц. Получены новые оценки билинейной сложности для ряда задач точного и приближенного умножения прямоугольных матриц. В том числе улучшена оценка граничного ранга для умножения матриц 3×3 и предложен практический алгоритм точного умножения квадратных матриц размерности n с асимптотической арифметической сложностью $O(n^{2.7743})$. Библ. 22. Табл. 6.

Ключевые слова: билинейная сложность, ранг задачи умножения матриц, граничный ранг, алгоритмы точного и приближенного умножения матриц, метод наименьших квадратов, целевая функция.

DOI: 10.7868/S0044466913120168

Прошло более 40 лет с того момента, как Фолькер Штрассен в 1969 г. своим неожиданным результатом (см. [1]) привлек внимание исследователей к задаче быстрого умножения больших матриц. Несмотря на усилия многих математиков и ряд заметных достижений, она донныне (2013 г.) не решена в общем виде. Не доказана и предполагаемая асимптотическая оценка сложности — $O(n^{2+\varepsilon})$, где $\varepsilon > 0$ — любое. В настоящее время лучшая теоретическая оценка показателя степени $p = 2.373$ получена на базе модификаций алгоритма (см. [2]), изначально давшего оценку $p = 2.376$. Столь малый прогресс позволяет говорить о “барьере Копперсмита-Винограда” в теоретической оценке сложности. Существенно и то, что данный алгоритм непрактичен в силу астрономической константы в оценке сложности. В работах, посвященных программной реализации практических алгоритмов быстрого умножения, используются алгоритмы с большими показателями степени. В [3] используется алгоритм из [1] с $p = 2.8074$. В [4] используется предложенный в [5] алгоритм с $p = 2.7760$ и установлено, что этот алгоритм не дает преимущества в скорости перед алгоритмом на основе [1] вплоть до $n = 4640$. Поскольку мы рассматриваем и билинейные алгоритмы приближенного умножения матриц для задач с произведением размерностей не более 6^3 , ориентиром для нас будет алгоритм приближенного умножения из [6] с $p = 2.7713$. В последние годы не отмечено существенного прогресса в исследованиях и замечен спад интереса математиков к этой непростой задаче. Вместе с тем в [7] “разработка алгоритмов умножения матриц с нижней возможной границей сложности” поставлена четвертой по важности задачей среди семи основных проблем вычислительной линейной алгебры. Подробный обзор работ до 1988 г. приведен в [8]. В этой работе незнакомый с проблемой читатель найдет определение билинейных алгоритмов, других используемых в статье терминов и доказательства некоторых свойств билинейных алгоритмов, кратко описанных ниже. Наиболее полный на сегодняшний день обзор литературы по теме можно найти в [9].

В данной статье автор приводит результаты своей попытки численного решения определяющей задачу системы нелинейных уравнений для малых размерностей матриц-сомножителей. Целью работы была оценка билинейных сложностей задач умножения, а также поиск более быстрых практических алгоритмов умножения матриц.

Под практическим алгоритмом далее понимается алгоритм быстрого умножения матриц, который может быть реализован на современных компьютерах, так что при умножении некоторых матриц число арифметических операций будет действительно меньше, чем при использовании классического метода. Так, определенная практичность отделяет класс рассматриваемых авто-

ром алгоритмов от сугубо теоретических конструкций астрономических размерностей и является необходимым, но не достаточным условием практической полезности алгоритма. Реализация практически полезных алгоритмов во всех ее аспектах в данной работе не рассматривается.

Все расчеты проводились с точностью до 15 десятичных цифр на обычном персональном компьютере.

Рассмотрим задачу умножения матрицы размерности $n_1 \times n_2$ на матрицу размерности $n_2 \times n_3$. Минимальное число активных умножений в некоммутативном алгоритме, необходимое для перемножения матриц $R = R(n_1, n_2, n_3)$, называется *билинейной сложностью* или *рангом задачи умножения матриц*. Известно, что если все три размерности больше 1, то $R < n_1 n_2 n_3$. Это наряду с тем, что алгоритм применим к любому некоммутативному кольцу элементов, включая матрицы, и позволяет строить алгоритмы быстрого умножения больших матриц (см. [1]).

Рассматриваемые в работе алгоритмы быстрого умножения больших матриц определяются алгоритмами умножения двух матриц малых конкретных размерностей $G = AB$ с малым числом активных умножений T ($R \leq T < n_1 n_2 n_3$). Эти алгоритмы задаются тремя соответствующими элементами матриц G, A, B множествами коэффициентов-множителей

$$\begin{aligned}\gamma_{i_1 i_2}^t, \quad i_1 = 1, 2, \dots, n_1, \quad i_2 = 1, 2, \dots, n_2, \quad t = 1, 2, \dots, T, \\ \alpha_{j_1 j_2}^t, \quad j_1 = 1, 2, \dots, n_1, \quad j_2 = 1, 2, \dots, n_2, \quad t = 1, 2, \dots, T, \\ \beta_{k_1 k_2}^t, \quad k_1 = 1, 2, \dots, n_2, \quad k_2 = 1, 2, \dots, n_3, \quad t = 1, 2, \dots, T,\end{aligned}$$

которые удовлетворяют системе уравнений (см. [10])

$$\sum_{t=1}^T \gamma_{i_1 i_2}^t \alpha_{j_1 j_2}^t \beta_{k_1 k_2}^t = L_{i_1 i_2 j_1 j_2 k_1 k_2}, \quad (1)$$

где

$$\begin{aligned}L_{i_1 i_2 j_1 j_2 k_1 k_2} &= \delta_{i_1 j_1} \delta_{j_2 k_1} \delta_{i_2 k_2}, \\ \delta_{ij} &= \begin{cases} 0, & i \neq j, \\ 1, & i = j. \end{cases}\end{aligned}$$

Минимальное $T = R$, при котором система (1) имеет решение, является рангом задачи умножения матриц. Нетрудно показать, что функция $R = R(n_1, n_2, n_3)$ симметрична и, следовательно, оценки билинейной сложности умножения матриц можно указывать только для задач (n_1, n_2, n_3) , в которых размерности упорядочены по неубыванию. Характеризуя решения задачи (1), помимо размерностей матриц будем указывать через точку с запятой число членов суммы в левой части (1), называемое *длиной решения*, не обязательно равное рангу. Далее билинейные алгоритмы умножения двух матриц конкретной размерности, исчерпывающим образом задаваемые корнями системы уравнений (1), в отличие от порождаемых ими алгоритмов умножения больших матриц будем называть *решениями*.

Известно, что билинейная сложность задачи (2, 2, 2) равна 7 (см. [11]), а задачи (2, 2, 3) равна 11 (см. [12]). Для умножения матриц 3×3 в [13], [14] предложены решения (3, 3, 3; 23) с числом ненулевых коэффициентов $\gamma_{i_1 i_2}^t, \alpha_{j_1 j_2}^t, \beta_{k_1 k_2}^t$, в совокупности равным 153. Для матриц 4×4 известно решение (4, 4, 4; 49), которое строится из решения (2, 2, 2; 7). Для матриц 5×5 в [15] предложено решение (5, 5, 5; 100).

Для системы (1) возможна ситуация, когда сумма квадратов разностей правых и левых частей S_1 может принимать значения, сколь угодно близкие к 0, но решения не существует. Минимальное число слагаемых в (1) $r = r(n_1, n_2, n_3)$, при котором это возможно, называется *билинейной сложностью задачи приближенного умножения матриц* или *граничным рангом*. Функциональным решением задачи приближенного умножения матриц назовем такую совокупность функций $\gamma_{i_1 i_2}^t(x), \alpha_{j_1 j_2}^t(x), \beta_{k_1 k_2}^t(x)$ действительного аргумента, для которой $S_1(x) = O(x)$.

В [16] показано, что $r(2, 2, 3) \leq 10$. В [6] приведено функциональное решение (3, 3, 3; 21) такое, что все функции являются одночленами рядов Лорана, и доказано, что $r(4, 4, 4) \leq 47$.

Можно показать (см. [8]), что из любых билинейных решений $(n_1, n_2, n_3; r_1)$ и $(m_1, m_2, m_3; r_2)$ строится билинейное решение с характеристиками $(n_1 m_1, n_2 m_2, n_3 m_3; r_1 r_2)$. Это решение назовем

композицией или произведением исходных решений. Из решения для умножения прямоугольных матриц нетрудно получить два родственных решения той же длины, циклически переставляя размерности. Если перемножить три таких решения, то из решения для прямоугольных матриц получим решение для квадратных матриц размерности $n = n_1 n_2 n_3$. Такая процедура симметризации возможна и для разных прямоугольных решений, например $(2, 2, 3; r_1)$ и $(3, 3, 2; r_2)$.

Очевидно, что объединение решений $(n_1, n_2, k; r_1)$ и $(n_1, n_2, m; r_2)$, соответствующее объединению вторых сомножителей по столбцам в матрицу большей размерности, дает решение вида $(n_1, n_2, k + m; r_1 + r_2)$, которое мы будем называть *суммой решений*. Пусть имеется решение $(n_1, 2, k; r_1)$ для задачи частичного умножения матриц (см. [6]), в которой обнулен элемент b_{1k} , и решение $(n_1, 2, m; r_2)$ для задачи, в которой обнулен элемент b_{21} . Нетрудно видеть, что эти решения можно просуммировать с наложением столбцов, включающих нулевые элементы, и получить решение $(n_1, 2, k + m - 1; r_1 + r_2)$ для задачи обычного умножения матриц. Такое сведение асимметричных задач к двум задачам меньшей размерности позволило в данной работе существенно улучшить оценки граничных рангов.

Билинейный алгоритм быстрого умножения матриц сводится к последовательному представлению больших матриц в виде матриц малой размерности из элементов-подматриц и рекурсивного использования для их умножения одного или нескольких решений небольшой размерности и малой длины.

Нетрудно видеть (см. [8]), что показатель степени в асимптотической оценке арифметической сложности $O(n^p)$ такого алгоритма умножения квадратных матриц, построенного из одного решения для прямоугольных матриц, вычисляется по формуле

$$p = 3 \log_{n_1 n_2 n_3}(r_1). \quad (2)$$

Далее для изложения результатов нам будет достаточно отмеченных выше свойств решений и операций с ними.

Отметим, что число уравнений системы (1) не совпадает с числом ее неизвестных. Заменяя парные индексы одинаковыми, запишем целевую функцию для этой системы уравнений в виде

$$S_1 = \sum_{i=1}^{N_1} \sum_{j=1}^{N_2} \sum_{k=1}^{N_3} \left(\sum_{t=1}^T \gamma_i^t \alpha_j^t \beta_k^t - L_{ijk} \right)^2, \quad (3)$$

где $N_1 = n_1 n_3$, $N_2 = n_1 n_2$, $N_3 = n_2 n_3$.

Для данной функции методом наименьших квадратов (МНК) (см. [17]) можно получить систему уравнений пятой степени. Эта система, как легко видеть, распадается на три линейные (матричные) подсистемы относительно каждой группы неизвестных коэффициентов. Решение каждой такой подсистемы является решением частной задачи минимизации целевой функции для соответствующей группы неизвестных при заданных двух других группах. Последовательное итеративное решение этих матричных подсистем является достаточно быстрым процессом, который не увеличивает целевую функцию и при определенных условиях сходится к некоторому решению.

Вычислительный эксперимент показывает, что локальные экстремумы и нижние грани целевой функции (3) для всех исследованных задач полного умножения матриц имеют целочисленные значения и, в частности, если удалось получить значение $S_1 < 1$, то в дальнейшем S_1 , по всей видимости, стремится к 0. Единственным препятствием этому является снижение точности вычислений вследствие роста корней по модулю для решений задачи приближенного умножения. Совокупность коэффициентов, удовлетворяющую последнему неравенству, если не найдено точное решение заданной длины, будем называть *численным решением задачи приближенного полного умножения матриц*. Для задач частичного умножения матриц нижние грани целевой функции могут быть не целочисленными. В этом случае не найдено предельных точек целевой функции больше 0 и меньше 3/4, что также позволяет гипотетически определять наличие приближенного решения.

При минимизации S_1 указанным способом и дальнейшем использовании результатов возникают четыре проблемы.

1. Неизвестные коэффициенты в процессе итераций могут неограниченно расти по модулю, даже в случае, когда точное решение заведомо существует. Рост коэффициентов по модулю быстро снижает точность решения системы уравнений МНК.

2. Для задачи точного умножения матриц более доказательными и практически полезными являются решения с малым числом различных значений, например такие, где все искомые коэффициенты принадлежат множеству $\{-1, 0, 1\}$, а итеративный процесс сам по себе не приводит к такому решению. Далее *дискретным* будем называть решение точной задачи умножения с не более чем 5 разными значениями коэффициентов.

3. Наличие численного решения задачи приближенного умножения позволяет только выдвинуть гипотезу о существовании приближенного функционального решения. Для доказательства оценки граничного ранга необходимо это функциональное решение построить.

4. При увеличении размерностей матриц процесс, как правило, начинает “застывать” в локальных целочисленных экстремумах, больших 0, и в тех случаях, когда решение заведомо существует.

Введем следующую целевую функцию, систему уравнений МНК для которой можно решать тем же быстрым методом, что и систему уравнений для функции (3):

$$S_2 = S_1 + \varepsilon \sum_{t=1}^T \left(\sum_{i=1}^{N_1} (\gamma_i^t - \tilde{\gamma}_i^t)^2 + \sum_{j=1}^{N_2} (\alpha_j^t - \tilde{\alpha}_j^t)^2 + \sum_{k=1}^{N_3} (\beta_k^t - \tilde{\beta}_k^t)^2 \right), \quad (4)$$

где ε — неотрицательное число, а $\tilde{\gamma}_i^t, \tilde{\alpha}_j^t, \tilde{\beta}_k^t$ — модельные коэффициенты. Выбирая значения этих параметров в процессе оптимизации, исследователь может в определенных пределах подгонять значения искомых коэффициентов к желаемому виду. Оказалось, что, применяя с помощью второго слагаемого (4) различные эвристики подбора, можно получать не только точные дискретные решения с числом разных значений коэффициентов, равным 3, но и строить функциональные приближенные решения.

Заметим, что при минимизации (4) условия

$$\begin{cases} \tilde{\gamma}_i^t = \gamma_i^t, & \text{если } |\gamma_i^t| \leq b_\gamma; \quad \tilde{\gamma}_i^t = -b_\gamma, & \text{если } \gamma_i^t < -b_\gamma; \quad \tilde{\gamma}_i^t = b_\gamma, & \text{если } \gamma_i^t > b_\gamma; \\ \tilde{\alpha}_j^t = \alpha_j^t, & \text{если } |\alpha_j^t| \leq b_\alpha; \quad \tilde{\alpha}_j^t = -b_\alpha, & \text{если } \alpha_j^t < -b_\alpha; \quad \tilde{\alpha}_j^t = b_\alpha, & \text{если } \alpha_j^t > b_\alpha; \\ \tilde{\beta}_k^t = \beta_k^t, & \text{если } |\beta_k^t| \leq b_\beta; \quad \tilde{\beta}_k^t = -b_\beta, & \text{если } \beta_k^t < -b_\beta; \quad \tilde{\beta}_k^t = b_\beta, & \text{если } \beta_k^t > b_\beta; \end{cases} \quad (5)$$

где $\gamma_i^t, \alpha_j^t, \beta_k^t$ — значения неизвестных, полученные на предыдущем шаге, а $b_\gamma, b_\alpha, b_\beta$ — неотрицательные числа, позволяют при постепенном увеличении ε ограничить значения корней системы заданными, но не слишком малыми интервалами.

Поскольку в S_1 входят только произведения трех коэффициентов, нетрудно видеть, что значение минимума целевой функции (4) при ограничениях (5) зависит от одного аргумента $b = (b_\gamma b_\alpha b_\beta)^{\frac{1}{3}}$. При поиске точных дискретных решений оказалось возможным всегда выбирать одинаковые границы. Разные границы $b_\gamma, b_\alpha, b_\beta$ необходимы для подбора некоторых функциональных решений.

Минимальные степени полиномов Лорана и, соответственно, $b_\gamma, b_\alpha, b_\beta$ при этом выбираются следующим образом. Минимизируя целевую функцию (4) для разных b при достаточно большом ε , экспериментально оцениваем величину q в выражении $S_1 = O(b^q)$. Если решение представимо полиномами Лорана, то $h = 6/q$ есть порядок приближенного разложения тензора умножения матриц, который определен в [18] и который мы назовем *порядком полиномиального решения*. Зная этот порядок, можно выбрать минимальные степени полиномов Лорана в каждой из трех групп коэффициентов. (Сумма этих степеней равна $-h$). Оказалось, что можно искать решения, в которых все полиномы с минимальной степенью в группе имеют вид $\pm x^p$, где p неположительные, вообще говоря, разные для разных групп. Вычисляя эти полиномы для некоторого x в $[0.05, 0.2]$, получаем границы $b_\gamma, b_\alpha, b_\beta$, позволяющие далее искать полиномиальное решение.

Следующий эвристический алгоритм позволяет дискретизировать решения задач точного умножения матриц, а также является третьим шагом построения функциональных решений для задач приближенного умножения матриц.

Задаем некоторое малое ε . Итеративно минимизируем (4) при условиях $b_\gamma = 0, b_\alpha = 0, b_\beta = 0$ в (5) до достижения стабильного значения целевой функции (4) меньше 1.

Задаем некоторые положительные $B_\gamma, B_\alpha, B_\beta$ и числа $M_\gamma, M_\alpha, M_\beta$, первоначально равные 1.

Упорядочиваем текущие значения искоемых коэффициентов каждой группы после каждой итерации по неубыванию модуля. Для множеств из $M_\gamma, M_\alpha, M_\beta$ наибольших по модулю коэффициентов выбираем $\tilde{\gamma}'_i, \tilde{\alpha}'_j, \tilde{\beta}'_k$ согласно (5) при $b_\gamma = B_\gamma, b_\alpha = B_\alpha, b_\beta = B_\beta$, для остальных выбираем $\tilde{\gamma}'_i, \tilde{\alpha}'_j, \tilde{\beta}'_k$ согласно (5) при $b_\gamma = 0, b_\alpha = 0, b_\beta = 0$.

Продолжая минимизацию (4), постепенно увеличиваем числа $M_\gamma, M_\alpha, M_\beta$ и ε так, чтобы целевая функция оставалась достаточно малой.

Этот процесс часто приводит либо к точному дискретному решению, либо к приближенному численному решению с большим числом нулевых коэффициентов и большим числом максимальных, совпадающих по модулю коэффициентов в каждой группе, которым соответствуют одночлены Лорана минимальной степени. Последний этап подбора полиномиального решения описан в конце статьи.

Ниже приведена табл. 1 минимальных длин решений, которые удалось получить автору. Они являются оценками сверху рангов задач точного и приближенного умножения матриц. Звездочкой отмечены оценки для задач точного умножения, решения которых не удалось привести к дискретному виду (в этом случае точным считалось решение, для которого $S_1 < 10^{-12}$ и все корни не превосходят 1 по модулю). В скобках указаны длины полученных приближенных численных решений, для которых пока не удалось подобрать функциональные.

Размер таблицы обусловлен двумя причинами. Для задач большой размерности метод не приводит к решениям точного вида, даже в случаях, когда такое решение заведомо существует. В частности, точное решение (4, 4, 4; 49) прямой оптимизацией получить не удалось несмотря на то, что удалось получить приближенное (4, 4, 4; 46). Для приближенных решений проблемы связаны с точностью и длительностью вычислений. При наличии более мощного компьютера эта часть таблицы более перспективна для улучшения и расширения.

Ряд аналогичных расчетов в комплексных числах не позволил улучшить результаты, полученные для действительных чисел.

Среди оценок точных рангов новыми являются оценки № 20, 21, 27, 32.

Среди оценок граничного ранга новыми являются все оценки, кроме № 1, 2, 3 и частично 5 (приближенное решение (2, 2, 4; 13) и приближенное решение (2, 2, 5; 16) для полей характеристики, отличной от 2, опубликованы в [19]).

Для задач вида $(2, n_2, n_3)$, все полученные длины решений удовлетворяют формуле

$$R(2, n_2, n_3) = \left\lceil \frac{(3n_2 + 1)n_3}{2} \right\rceil,$$

которая является известной оценкой сверху (см. [20]).

При $n_2 \geq 3$ табличные оценки $R(n_2, n_2, n_3)$ можно сопоставить с теоретической оценкой билинейной сложности умножения прямоугольных матриц снизу $R = 2n_3n_2 + 2n_2 - n_3 - 2$ (см. [21]). Отношения табличных оценок сверху к теоретическим оценкам снизу для задач $(3, 3, n_3)$ близки к 1.2.

В последних двух равенствах аргументы упорядочены по неубыванию.

Заметим, что умножение матриц 54×54 представимо как умножение матрицы 3×3 на матрицу 3×6 соответственно из подматриц 3×6 и 6×3 соответственно из подматриц 6×3 и 3×3 . Для композиционного решения $(54, 54, 54; 40^3)$ в асимптотической оценке арифметической сложности показатель степени $p = 3\log_{54}(40) = 2.7743$. Таким образом, решение $(3, 3, 6, 40)$ дает нам практический алгоритм точного умножения квадратных матриц асимптотически более быстрый, чем известные практические алгоритмы точного умножения. В построении композиционного решения нет необходимости, поскольку алгоритм умножения прямоугольных матриц можно применять непосредственно, разбивая умножаемые квадратные матрицы на подматрицы-элементы указанным выше образом, и это дает большую простоту, эффективность и гибкость в сочетании с другими алгоритмами. Решение $(3, 3, 6; 40)$ приведено ниже в табл. 2.

Два других решения, соответствующие перестановкам размерностей, получаются путем соответствующих перестановок групп (колонок) матриц коэффициентов и их транспонирования. При перестановке колонок сомножителя и результата надо транспонировать матрицы коэффициентов непереставляемого сомножителя. При перестановке колонок сомножителей надо транспонировать все матрицы коэффициентов.

Таблица 1

№	Размерности задачи			$n_1 n_2 n_3$	Оценка ранга точного умно- жения R	Оценка ранга приближенного умножения r
	n_1	n_2	n_3			
1	2	2	2	8	7	7
2	2	2	3	12	11	10
3	2	2	4	16	14	13
4	2	3	3	18	15	14
5	2	2	5	20	18	16
6	2	3	4	24	20	19
7	2	2	6	24	21	20 (19)
8	3	3	3	27	23	20
9	2	2	7	28	25	23 (22)
10	2	3	5	30	25	23
11	2	4	4	32	26	24
12	2	2	8	32	28	26
13	3	3	4	36	29	27
14	2	3	6	36	30	28
15	2	2	9	36	32	29
16	2	4	5	40	33	32 (30)
17	2	2	10	40	35	32
18	2	3	7	42	35	33 (32)
19	2	2	11	44	39	36 (35)
20	3	3	5	45	36	34
21	3	4	4	48	38	35
22	2	4	6	48	39	37 (36)
23	2	3	8	48	40	37
24	2	2	12	48	42	39 (38)
25	2	5	5	50	40	39 (38)
26	2	2	13	52	46	43 (41)
27	3	3	6	54	40	39
28	2	3	9	54	45	42
29	2	4	7	56	46	43
30	2	2	14	56	49	46 (44)
31	2	5	6	60	48*	46
32	3	4	5	60	48	46 (45)
33	2	3	10	60	50	47 (46)
34	2	2	15	60	53	49 (48)
35	3	3	7	63	49	47
36	4	4	4	64	49	46
37	2	4	8	64	52*	48
38	2	2	16	64	56	52 (51)

Для данной задачи существует родственное решение со значениями в множестве $\{-1/2, 0, 1/2\}$ и с тем же числом нулевых коэффициентов. В отличие от остальных задач, для которых найдены дискретные решения, решение (3, 3, 6; 40) не удалось привести к виду со значениями в множестве $\{-1, 0, 1\}$. Суммируя это решение, а потом результат суммирования с собой по размерности 3, получаем, что $R(6, 6, 6) \leq 160$.

Таблица 2. Алгоритм точного умножения матрицы 3×3 на матрицу 3×6 с 40 активными умножениями

t	γ^t						α^t			β^t					
1	-1/8	-1	0	0	0	1	1	0	1	0	-1/8	0	0	1	1/8
	-1/8	0	1	1	0	0	0	1	1	0	0	1/8	1/8	-1	0
	0	1	0	1	-1/8	0	-1	1	0	-1	0	1/8	0	0	1/8
2	1/8	0	-1	-1	0	0	1	0	-1	0	0	-1/8	-1/8	-1	0
	1/8	1	0	0	0	-1	0	1	-1	0	1/8	0	0	1	-1/8
	0	0	1	0	1/8	-1	-1	1	0	-1	-1/8	0	1/8	0	0
3	0	0	-1	0	-1/8	1	-1	0	-1	-1	0	1/8	0	0	-1/8
	0	1	0	1	-1/8	0	0	1	-1	-1	1/8	0	1/8	0	0
	-1/8	0	1	1	0	0	1	1	0	0	-1/8	0	0	1	-1/8
4	-1/8	1	0	0	0	1	-1	0	1	0	-1/8	0	0	-1	-1/8
	1/8	0	1	-1	0	0	0	-1	-1	0	0	-1/8	1/8	-1	0
	0	-1	0	1	-1/8	0	1	1	0	-1	0	-1/8	0	0	1/8
5	-1/8	0	-1	1	0	0	1	0	1	0	0	-1/8	1/8	1	0
	1/8	-1	0	0	0	-1	0	1	-1	0	-1/8	0	0	1	-1/8
	0	0	1	0	-1/8	1	-1	-1	0	-1	1/8	0	1/8	0	0
6	0	1	0	-1	-1/8	0	-1	0	1	-1	-1/8	0	1/8	0	0
	0	0	1	0	1/8	1	0	1	-1	1	0	1/8	0	0	1/8
	1/8	1	0	0	0	1	-1	1	0	0	0	-1/8	-1/8	-1	0
7	0	0	-1	0	-1/8	-1	-1	0	-1	-1	0	1/8	0	0	1/8
	0	-1	0	1	1/8	0	0	1	1	1	-1/8	0	1/8	0	0
	1/8	0	-1	1	0	0	-1	1	0	0	-1/8	0	0	1	1/8
8	1/8	1	0	0	0	1	1	0	1	0	1/8	0	0	-1	1/8
	-1/8	0	1	-1	0	0	0	1	-1	0	0	1/8	-1/8	-1	0
	0	1	0	-1	-1/8	0	1	1	0	1	0	-1/8	0	0	1/8
9	0	0	1	0	-1/8	1	-1	0	1	-1	0	-1/8	0	0	-1/8
	0	1	0	-1	1/8	0	0	1	-1	1	1/8	0	-1/8	0	0
	-1/8	0	-1	1	0	0	1	-1	0	0	-1/8	0	0	-1	1/8
10	0	-1	0	1	-1/8	0	-1	0	-1	-1	1/8	0	-1/8	0	0
	0	0	-1	0	1/8	-1	0	1	1	1	0	-1/8	0	0	-1/8
	-1/8	1	0	0	0	1	1	-1	0	0	0	-1/8	-1/8	1	0
11	0	1	0	1	1/8	0	1	0	1	-1	1/8	0	1/8	0	0
	0	0	-1	0	1/8	1	0	-1	1	-1	0	1/8	0	0	-1/8
	-1/8	1	0	0	0	-1	1	1	0	0	0	-1/8	1/8	1	0
12	-1/8	0	-1	-1	0	0	1	0	1	0	0	-1/8	-1/8	1	0
	-1/8	1	0	0	0	-1	0	1	1	0	1/8	0	0	-1	-1/8
	0	0	-1	0	1/8	1	1	-1	0	-1	1/8	0	-1/8	0	0
13	-1/8	1	0	0	0	-1	-1	0	1	0	-1/8	0	0	-1	1/8
	-1/8	0	-1	-1	0	0	0	-1	1	0	0	1/8	1/8	1	0
	0	1	0	1	1/8	0	-1	1	0	-1	0	-1/8	0	0	-1/8
14	0	0	-1	0	1/8	1	-1	0	1	1	0	1/8	0	0	-1/8
	0	1	0	1	1/8	0	0	1	1	1	1/8	0	1/8	0	0
	-1/8	0	-1	-1	0	0	-1	-1	0	0	1/8	0	0	1	1/8

Таблица 2. Продолжение

t	γ^t						α^t			β^t					
15	0	-1	0	-1	1/8	0	-1	0	1	1	1/8	0	1/8	0	0
	0	0	1	0	1/8	-1	0	1	1	1	0	1/8	0	0	-1/8
	1/8	1	0	0	0	-1	1	1	0	0	0	1/8	-1/8	1	0
16	-1/8	0	1	-1	0	0	1	0	-1	0	0	1/8	-1/8	1	0
	1/8	1	0	0	0	1	0	1	1	0	1/8	0	0	1	1/8
	0	0	1	0	1/8	1	1	1	0	1	1/8	0	1/8	0	0
17	-1/8	0	0	0	-1/8	0	0	0	0	-1	-1/8	1/8	-1/8	1	-1/8
	1/8	1	-1	1	-1/8	1	-1	-1	0	-1	0	0	0	1	0
	0	0	0	1	0	1	-1	-1	0	0	0	0	1/8	0	-1/8
18	-1/8	1	-1	1	-1/8	1	1	1	0	-1	0	0	0	-1	0
	1/8	0	0	0	-1/8	0	0	0	0	-1	1/8	-1/8	1/8	-1	1/8
	0	-1	1	0	0	0	-1	-1	0	0	-1/8	-1/8	0	0	0
19	1/8	0	0	0	1/8	0	0	0	0	-1	-1/8	1/8	1/8	1	1/8
	1/8	1	-1	-1	-1/8	-1	-1	1	0	1	0	0	0	-1	0
	0	0	0	-1	0	-1	-1	1	0	0	0	0	-1/8	0	1/8
20	1/8	1	1	1	-1/8	-1	0	1	1	1	0	0	0	1	0
	1/8	0	0	0	-1/8	0	0	1	1	1	1/8	1/8	1/8	-1	-1/8
	0	-1	1	0	0	0	0	0	0	0	1/8	1/8	0	0	0
21	-1/8	0	0	0	-1/8	0	-1	0	1	1	1/8	1/8	-1/8	1	1/8
	1/8	1	1	-1	1/8	1	1	0	-1	-1	0	0	0	1	0
	0	0	0	1	0	1	0	0	0	0	0	0	1/8	0	-1/8
22	-1/8	0	0	0	-1/8	0	-1	0	-1	1	-1/8	-1/8	-1/8	1	1/8
	-1/8	1	1	1	-1/8	-1	-1	0	-1	1	0	0	0	-1	0
	0	0	0	1	0	1	0	0	0	0	0	0	-1/8	0	1/8
23	0	1	1	0	0	0	0	0	0	0	0	0	1/8	0	1/8
	0	0	0	-1	0	1	0	-1	1	0	-1/8	1/8	0	0	0
	1/8	1	-1	-1	-1/8	1	0	-1	1	1	1/8	-1/8	-1/8	-1	1/8
24	0	-1	-1	0	0	0	0	0	0	0	0	0	1/8	0	1/8
	0	0	0	1	0	-1	0	-1	-1	0	-1/8	1/8	0	0	0
	1/8	1	-1	1	-1/8	-1	0	-1	-1	-1	-1/8	1/8	-1/8	1	1/8
25	-1/8	-1	1	1	-1/8	1	-1	1	0	1	0	0	0	1	0
	-1/8	0	0	0	1/8	0	0	0	0	-1	-1/8	1/8	1/8	-1	1/8
	0	1	-1	0	0	0	1	-1	0	0	1/8	1/8	0	0	0
26	-1/8	-1	-1	1	1/8	-1	0	-1	1	-1	0	0	0	-1	0
	1/8	0	0	0	-1/8	0	0	1	-1	1	1/8	1/8	-1/8	-1	1/8
	0	-1	1	0	0	0	0	0	0	0	-1/8	-1/8	0	0	0
27	0	1	1	0	0	0	1	0	1	0	0	0	1/8	0	1/8
	0	0	0	-1	0	1	0	0	0	0	-1/8	1/8	0	0	0
	1/8	-1	-1	-1	1/8	-1	-1	0	-1	-1	1/8	1/8	1/8	-1	1/8
28	0	1	1	0	0	0	-1	0	-1	0	0	0	1/8	0	1/8
	0	0	0	-1	0	1	0	0	0	0	-1/8	1/8	0	0	0
	-1/8	1	1	-1	-1/8	-1	-1	0	-1	1	-1/8	-1/8	1/8	1	1/8

Таблица 2. Окончание

t	γ^t						α^t			β^t					
29	0	1	1	0	0	0	1	0	-1	0	0	0	1/8	0	1/8
	0	0	0	-1	0	1	0	0	0	0	-1/8	1/8	0	0	0
	1/8	1	1	1	1/8	1	1	0	-1	-1	-1/8	-1/8	-1/8	-1	-1/8
30	0	1	1	0	0	0	0	0	0	0	0	0	1/8	0	1/8
	0	0	0	-1	0	1	0	1	-1	0	-1/8	1/8	0	0	0
	-1/8	1	-1	1	1/8	-1	0	-1	1	-1	1/8	-1/8	1/8	1	-1/8
31	0	1	1	0	0	0	0	0	0	0	0	0	1/8	0	1/8
	0	0	0	-1	0	1	0	-1	-1	0	-1/8	1/8	0	0	0
	1/8	-1	1	1	-1/8	-1	0	1	1	1	-1/8	1/8	1/8	-1	-1/8
32	0	-1	-1	0	0	0	1	0	-1	0	0	0	1/8	0	1/8
	0	0	0	1	0	-1	0	0	0	0	-1/8	1/8	0	0	0
	1/8	1	1	-1	1/8	-1	-1	0	1	1	1/8	1/8	-1/8	1	-1/8
33	-1/8	1	-1	-1	-1/8	-1	1	-1	0	-1	0	0	0	-1	0
	-1/8	0	0	0	1/8	0	0	0	0	1	-1/8	1/8	1/8	1	1/8
	0	1	-1	0	0	0	1	-1	0	0	-1/8	-1/8	0	0	0
34	1/8	0	0	0	1/8	0	1	0	-1	1	1/8	1/8	1/8	1	-1/8
	1/8	1	1	1	1/8	-1	1	0	-1	1	0	0	0	-1	0
	0	0	0	-1	0	-1	0	0	0	0	0	0	-1/8	0	1/8
35	-1/8	1	1	1	1/8	-1	0	1	-1	-1	0	0	0	-1	0
	-1/8	0	0	0	1/8	0	0	1	-1	-1	1/8	1/8	1/8	1	-1/8
	0	1	-1	0	0	0	0	0	0	0	-1/8	-1/8	0	0	0
36	-1/8	-1	1	-1	-1/8	-1	1	1	0	-1	0	0	0	-1	0
	1/8	0	0	0	-1/8	0	0	0	0	-1	-1/8	1/8	-1/8	-1	-1/8
	0	-1	1	0	0	0	1	1	0	0	-1/8	-1/8	0	0	0
37	1/8	0	0	0	1/8	0	0	0	0	-1	1/8	-1/8	-1/8	1	-1/8
	1/8	-1	1	1	-1/8	1	-1	1	0	1	0	0	0	-1	0
	0	0	0	-1	0	-1	1	-1	0	0	0	0	-1/8	0	1/8
38	1/8	0	0	0	1/8	0	0	0	0	1	-1/8	1/8	-1/8	-1	-1/8
	-1/8	1	-1	1	1/8	1	-1	-1	0	1	0	0	0	-1	0
	0	0	0	-1	0	-1	1	1	0	0	0	0	-1/8	0	1/8
39	1/8	-1	-1	1	-1/8	-1	0	-1	-1	1	0	0	0	1	0
	-1/8	0	0	0	1/8	0	0	1	1	-1	1/8	1/8	-1/8	1	1/8
	0	1	-1	0	0	0	0	0	0	0	1/8	1/8	0	0	0
40	1/8	0	0	0	1/8	0	-1	0	-1	-1	1/8	1/8	-1/8	-1	1/8
	-1/8	1	1	-1	-1/8	1	1	0	1	1	0	0	0	-1	0
	0	0	0	-1	0	-1	0	0	0	0	0	0	-1/8	0	1/8

В точном решении (3, 3, 3; 23) удалось уменьшить количество ненулевых коэффициентов. В приведенном ниже алгоритме число ненулевых коэффициентов меньше, чем в решениях [13], [14]. Матрицы коэффициентов развернуты по строкам (см. табл. 3).

С точки зрения “практического программирования” приведенные решения являются лишь заготовками. Нетрудно увидеть в представленных линейных формах каждой из групп совпадающие подмножества двух и более ненулевых коэффициентов. Таким образом, необходима опти-

Таблица 3. Алгоритм точного умножения матриц 3×3 с 23 активными умножениями и числом ненулевых коэффициентов, равным 139

t	γ^t (ненулевых 45)	α^t (ненулевых 49)	β^t (ненулевых 45)
1	-1 0 0 1 0 0 1 0 0	0 0 0 0 0 0 0 0 1	-1 0 0 -1 0 0 1 0 0
2	0 0 0 -1 0 0 0 1 0	0 0 0 0 1 0 0 0 1	0 0 0 -1 0 0 0 1 0
3	0 -1 1 0 0 0 0 0 0	1 1 0 0 0 0 1 1 0	0 0 0 0 0 1 0 0 0
4	0 0 0 0 0 1 0 0 0	-1 0 0 1 0 0 0 0 0	0 1 1 0 0 0 0 0 0
5	0 -1 1 0 0 0 0 1 -1	0 0 0 0 0 0 -1 -1 0	0 -1 0 0 0 0 0 0 0
6	0 1 0 1 1 0 0 0 0	0 0 0 0 1 1 0 0 0	0 0 0 0 0 0 0 1 0
7	0 0 0 -1 0 0 0 0 0	0 0 0 -1 0 0 0 0 -1	1 0 0 0 0 0 0 0 0
8	0 0 0 0 0 0 1 0 -1	0 0 0 0 0 0 1 0 1	1 0 0 0 0 0 0 0 0
9	0 0 0 0 1 -1 0 1 0	0 0 0 0 1 0 0 0 0	0 -1 0 0 1 0 0 -1 0
10	0 0 0 0 0 0 0 1 0	0 0 0 0 1 0 0 -1 0	0 1 0 1 -1 0 0 0 0
11	0 0 0 0 0 0 -1 -1 0	0 0 0 0 0 0 0 -1 -1	0 0 0 1 0 0 0 0 0
12	0 1 0 0 0 1 0 0 0	0 0 -1 0 1 1 0 0 0	0 0 0 0 1 1 0 -1 0
13	1 0 0 0 0 0 0 0 0	0 0 1 0 0 0 0 0 1	0 0 0 0 0 0 1 0 1
14	0 0 0 0 0 0 0 0 1	1 0 0 0 0 0 1 0 0	1 0 1 0 0 -1 0 0 0
15	-1 0 0 0 0 0 0 0 1	-1 0 0 0 0 0 0 0 1	1 0 0 0 0 0 0 0 1
16	0 0 0 0 0 1 0 0 0	0 0 -1 0 0 1 0 0 0	0 0 0 0 -1 -1 0 1 1
17	0 1 0 0 0 0 0 0 0	0 -1 -1 0 1 1 0 0 0	0 0 0 0 -1 -1 0 0 0
18	0 0 0 1 0 0 0 0 0	0 0 0 0 0 -1 0 0 1	0 0 0 0 0 0 -1 1 0
19	0 1 -1 0 0 0 0 0 1	-1 0 0 0 0 0 -1 -1 0	0 -1 0 0 0 -1 0 0 0
20	1 0 -1 0 0 -1 0 0 0	-1 0 -1 0 0 0 0 0 0	0 0 0 0 0 0 0 0 1
21	0 0 -1 0 0 -1 0 0 1	1 0 0 0 0 0 0 0 0	0 -1 -1 0 0 0 0 0 1
22	0 0 0 0 -1 1 0 0 0	0 0 0 -1 -1 0 0 0 0	0 1 0 0 0 0 0 0 0
23	-1 0 0 0 0 0 0 0 0	0 -1 0 0 0 0 0 0 1	0 0 0 1 0 0 0 0 0

мизация вычислений, которая уменьшит константу алгоритмов. Кроме того, следует оценить устойчивость алгоритмов по методике из [22].

Тривиальная оптимизация за счет устранения дублирующих вычислений позволяет снизить число неактивных операций для решения $(3, 3, 6; 40)$ с 1246 до 353 (с учетом делений на 8), а для $(3, 3, 3; 23)$ — с 84 до 64. Предварительные оценки позволяют предположить, что наиболее быстрым практическим алгоритмом будет составной алгоритм, использующий разные решения в ходе декомпозиции и уменьшения размерностей перемножаемых подматриц. Например, $(3, 3, 6; 40)$, $(2, 2, 2; 7)$, $(3, 3, 3; 23)$, $(n_1, n_2, n_3; n_1 n_2 n_3)$. При оптимальном подборе числа шагов для использования каждого решения такая схема должна быть быстрее известных на практическом отрезке размерностей матриц. Существенно, что для асимметричных решений, например $(3, 3, 6; 40)$, мы не обязаны использовать для симметризации все три шага. Симметризацию можно осуществить на заключительном этапе с помощью обычного умножения прямоугольных матриц или перед заключительным этапом с помощью других “малоразмерных” асимметричных решений, что может оказаться более выгодным.

Перейдем к алгоритмам приближенного умножения матриц.

Из № 8 табл. 1 вытекает, что $r(3, 3, 3) \leq 20$.

Из № 36 табл. 1 вытекает, что $r(4, 4, 4) \leq 46$. Функциональное решение $(4, 4, 4; 46)$ представляет собой упорядоченную совокупность 2208 функций, из которых 467 — нетождественные нули. Поскольку анализ таких данных без компьютера затруднителен, то во избежание ошибок автор полагает разумным не приводить этот и некоторые другие алгоритмы, но готов предоставить их желающим в электронном виде.

Из №№ 4, 8, 13, 21 табл. 1 вытекает существование нескольких алгоритмов приближенного умножения матриц, асимптотически более быстрых, чем известный алгоритм $(3, 3, 3; 21)$ из [6].

Таблица 4. Алгоритм приближенного умножения матрицы 3×2 на матрицу 2×3 с 14 активными умножениями

t	γ'			α'		β'		
1	0	-1	1	0	x^{-1}	0	x^{-1}	0
	-1	1	$-x^{-1}$	x	0	0	0	0
	x	$-x^{-1}$	x^{-1}	0	0			
2	x	0	$-x^2$	0	x^{-1}	x^{-1}	0	0
	0	0	x^{-1}	0	0	0	0	0
	x^{-1}	-1	$-x^{-1}$	x^2	-1			
3	$-x^2$	0	x^2	$-x^{-1}$	0	x^{-1}	0	x
	0	0	0	0	0	0	0	$-x$
	0	0	x^{-1}	$1 + x^2$	0			
4	0	0	0	0	0	1	1	x
	1	0	x^{-1}	1	0	0	0	0
	$-x$	-1	0	0	0			
5	0	0	x^2	$-x^{-1}$	$-x^{-1}$	$-x^{-1}$	0	0
	0	0	0	0	0	0	0	x
	0	0	x^{-1}	1	1			
6	0	0	0	0	$-x^{-1}$	$-x^{-1}$	0	0
	0	0	$-x^{-1}$	$-x$	0	0	1	0
	0	1	0	$-x^2$	1			
7	x	0	0	0	$-x^{-1}$	x^{-1}	0	0
	0	0	0	x^3	0	x	0	0
	x^{-1}	0	0	0	1			
8	0	0	0	0	$-x^{-1}$	0	$-x^{-1}$	0
	1	-1	x^{-1}	0	0	x^{-1}	0	0
	$-x$	0	0	0	0			
9	0	0	0	0	x^{-1}	0	0	0
	0	0	$-x^{-1}$	x	-1	0	1	0
	0	0	0	x^2	-1			
10	0	x^2	1	x^{-1}	0	0	x^{-1}	x
	0	0	0	0	0	0	0	$-x$
	0	0	x^{-1}	0	0			
11	0	0	1	x^{-1}	x^{-1}	0	$-x^{-1}$	0
	0	0	0	x	0	0	0	x
	0	0	x^{-1}	0	0			
12	$x^2 + x^3$	0	0	0	x^{-1}	0	0	0
	1	-1	x^{-1}	0	x	x^{-1}	0	0
	0	0	0	0	0			
13	0	1	0	0	x^{-1}	0	x^{-1}	0
	0	0	0	$x + x^2$	0	0	x	0
	0	x^{-1}	0	x^2	0			
14	0	0	0	0	0	0	0	0
	0	-1	x^{-1}	0	1	-1	-1	x
	0	0	0	0	0			

Таблица 5. Алгоритм приближенного умножения матрицы 2×2 на матрицу 2×3 с обнуленным a_{23} , требующий 8 активных умножений

t	γ^t			α^t		β^t		
1	$1-x$ x	x^{-1} 0	0 $-x^2$	1 x^{-1}	x 0	1 0	0 0	$-x^{-1}$ 0
2	1 0	x^{-1} 0	0 0	0 x^{-1}	x 0	-1 x^{-1}	$-x^2$ 0	x^{-1} 0
3	0 0	x^{-1} $-x^{-1}$	0 0	x^{-1} x^{-1}	0 0	$-x$ x^{-1}	0 -1	1 0
4	0 1	0 x^{-1}	0 0	0 x^{-1}	0 $-x$	$x+x^2$ $-x^{-1}$	x^2 0	0 0
5	-1 1	$-x^{-1}$ x^{-1}	0 0	0 x^{-1}	0 0	$-x$ x^{-1}	0 0	1 0
6	$1-x$ x	0 x^{-1}	x $-x^2$	x^{-1} 0	0 0	x^2 0	0 0	1 0
7	0 0	0 x^{-1}	0 0	$-x^{-1}$ $-x^{-1}$	0 x	$x+x^2$ $-x^{-1}$	0 1	0 0
8	0 0	x^{-1} 0	0 0	x^{-1} x^{-1}	x 0	0 $-x^{-1}$	x^2 1	0 0

Алгоритмы умножения квадратных матриц получаются из алгоритмов умножения прямоугольных матриц методом симметризации, описанным выше.

Решение $(2, 2, 4; 13)$ над любым полем и решение $(2, 2, 5; 16)$ над любым полем характеристики, отличной от 2, уже опубликованы. Поэтому приведем в функциональной форме решения $(2, 3, 3; 14)$, $(2, 2, 5; 16)$ и $(3, 3, 3; 20)$ над любым полем (см. табл. 4, 5 и 6).

Для решения табл. 4 $S_1 = 7x^2 + O(x^3) = O(b^{-2})$. Это решение имеет третий полиномиальный порядок.

В табл. 5 приведена первая половина решения $(2, 2, 5; 16)$, которая является решением $(2, 2, 3; 8)$ для задачи частичного умножения матриц с обнуленным элементом a_{23} . Вторая половина получается из приведенного решения допустимыми в (1) одновременными перестановками строк (столбцов) матриц решения, соответствующими перемещению элемента a_{23} в a_{12} и последующим сдвигом матриц α^t, β^t .

Для решения табл. 5 $S_1 = 7x^2 + O(x^3) = O(b^{-2})$. Это решение имеет третий полиномиальный порядок. Для полей характеристики, отличной от 2, существует решение второго порядка.

Для решения табл. 6 $S_1 = 14x^2 + O(x^3) = O(b^{-1})$. Это решение имеет шестой полиномиальный порядок.

Большинство функциональных алгоритмов было получено из численных решений задач приближенного умножения матриц эвристической подгонкой значений числовых коэффициентов к значениям полиномов Лорана от аргумента $x = 0.1$. Допустимыми коэффициентами полиномов считались целые числа по модулю, меньшие 5, а в отдельных случаях — дробь $1/2$, что обеспечивает однозначное соответствие полиномов числовым коэффициентам. Подгонка осуществлялась в процессе оптимизации по формуле (4). Численные коэффициенты решения, представляющие полиномы, делились на свободные — ограниченные только по модулю согласно (5) и фиксированные конкретными значениями $\tilde{\gamma}_i^t, \tilde{\alpha}_j^t, \tilde{\beta}_k^t$. При выборе и фиксации значения очередного коэффициента проверялось воздействие этого выбора на значение целевой функции. Уменьшение ε в (4) позволяет быстро оптимизировать значения оставшихся свободных корней,

Таблица 6. Алгоритм приближенного умножения матриц 3×3 с 20 активными умножениями

t	γ^t			α^t			β^t		
1	0	x^{-2}	0	x	0	$-x^3$	1	x	0
	$-x^2$	0	0	0	0	0	x^{-1}	0	0
	x^4	0	0	x^{-1}	0	x^{-2}	x^{-2}	0	$2x^3$
2	$-x$	0	x^{-2}	0	0	x^3	0	0	0
	$-x$	1	x	x^{-2}	$-x^{-1}$	0	0	0	x^3
	0	0	-1	0	0	$-x$	0	0	x^{-1}
3	x	0	$-x^{-2}$	0	0	0	0	0	0
	$x - x^4$	0	0	$-x^{-1}$	1	0	x^{-1}	1	$-x^2$
	0	0	1	0	$-x^{-2}$	0	0	0	0
4	$-x^{-1}$	x^{-2}	0	-1	x	0	x	0	0
	0	0	0	0	0	0	1	0	0
	x^4	0	0	$-x^{-2}$	x^{-1}	0	x^{-1}	0	0
5	0	$-x^{-2}$	0	0	0	$-x^3$	1	x	0
	x^2	0	0	x^{-2}	0	0	x^{-1}	0	0
	0	0	0	0	0	x^{-2}	x^{-2}	x^{-1}	x^3
6	1	-1	$-x^{-2}$	0	0	0	0	0	x^2
	0	0	0	0	0	0	x^{-2}	0	0
	$-x^2 - x^3$	x^2	1	x^{-2}	$-x^{-1}$	0	0	0	0
7	$-x^{-1}$	0	0	x	$-2x^2$	0	x	0	x^3
	0	0	0	0	0	0	x^{-1}	0	0
	x	0	0	x^{-2}	$-x^{-1}$	0	0	0	0
8	0	0	x^{-2}	x	0	$-x^3$	0	0	x
	$-x + x^2$	0	0	$-x^{-2}$	0	1	0	0	0
	0	0	0	x^{-1}	0	0	x^{-2}	0	0
9	$-x$	0	x^{-2}	0	0	0	0	0	0
	$-x$	$1 - x^3$	0	$-x^{-2}$	x^{-1}	0	0	x	0
	0	0	-1	0	0	0	0	0	x^{-1}
10	x	-1	$-x^{-2}$	0	0	0	0	0	x^3
	x	0	0	0	0	0	x^{-1}	1	0
	0	x^2	1	0	x^{-2}	0	0	0	0
11	0	0	x^{-2}	0	0	0	0	0	x
	$-x$	0	x	x^{-2}	0	-1	0	0	1
	0	0	0	0	0	0	0	1	0
12	0	0	$-x^{-2}$	0	x^2	0	0	0	0
	x	0	0	$-x^{-2}$	0	1	0	0	-1
	0	0	0	0	1	0	x^{-2}	0	0
13	$-x$	0	x^{-2}	0	0	0	0	0	0
	x^4	0	0	-1	x	0	x^{-2}	0	0
	0	0	-1	x^{-2}	$-x^{-1}$	0	0	0	0
14	x^{-1}	0	x^{-2}	-1	x	x^2	0	0	0
	$-x$	0	0	0	0	0	0	0	0
	0	0	0	$-x^{-2}$	x^{-1}	0	x^{-1}	0	0

Таблица 6. Окончание

t	γ^t			α^t			β^t		
15	0	x^{-1}	$-x^{-2}$	0	0	0	0	0	0
	$-x^3$	-1	$-x$	0	0	0	0	0	0
	0	x^4	0	0	0	x^{-2}	0	x^{-2}	$-x^2$
16	0	x^{-2}	0	0	$-x^2$	0	1	0	0
	0	0	0	x^{-2}	0	0	x^{-1}	-1	0
	0	0	0	0	-1	0	x^{-2}	0	0
17	0	x^{-2}	0	0	0	0	0	$-x$	x^3
	0	0	0	0	0	0	x^{-1}	0	0
	x	-1	0	x^{-1}	0	0	0	0	$-2x^3$
18	0	0	0	0	0	0	0	0	0
	0	x^2	1	$-x^{-1}$	1	x	0	0	1
	0	0	0	0	0	0	0	0	x^{-1}
19	0	0	x^{-2}	0	0	0	0	0	0
	0	1	x	-1	0	x^2	0	0	0
	0	0	0	0	0	x^{-2}	0	x^{-2}	0
20	0	x^{-2}	0	0	0	0	0	x	0
	0	x	0	x^{-2}	0	0	0	1	0
	0	0	0	0	0	0	0	x^{-1}	x^3

а увеличение ε позволяет проверить допустимость выбора при строгой фиксации уже отобранных коэффициентов. Если при завершении подбора удастся сохранить достаточно малое значение целевой функции, то итоговая полиномиальная целевая функция начинается с квадрата.

Используя периодический пересчет уже найденных полиномов и, как следствие, границ для разных значений x , можно построить и более тонкий метод подбора уже непосредственно полиномов. Это оказывается необходимым при наличии в решении большого числа двучленов.

СПИСОК ЛИТЕРАТУРЫ

1. Strassen V. Gaussian elimination is not optimal // Numer. Math. 1969. V. 13. P. 354–356.
2. Coppersmith D., Winograd S. Matrix multiplication via arithmetic progressions // J. Symbolic Comput. 1990. V. 9. P. 251–280.
3. Hunold S., Rauber T., Runger G. Combining building blocks for parallel multi-level matrix multiplication // Parallel comput. 2008. V. 34. Iss. 6–8. P. 411–426.
4. Kaporin I. The aggregation and cancellation techniques as a practical tool for faster matrix multiplication // Theor. Comput. Sci. 2004. V. 315. Iss. 2–3. P. 469–510.
5. Laderman J., Pan V.Y., Sha X.-H. On practical algorithms for accelerated matrix multiplication // Linear Algebra Appls. 1992. V. 162–164. P. 557–588.
6. Schönhage A. Partial and total matrix multiplication // SIAM J. Comput. 1981. V. 10. №. 3. P. 434–455.
7. Sen S.K. Open problems in computational linear algebra // Nonlinear Anal. 2005. V. 63. Iss. 5–7. P. 926–934.
8. Сложность умножения матриц. Обзор // Кибернетич. сборник. 1988. Вып. 25. С. 139–236.
9. Landsberg J. M. Geometry and the complexity of matrix multiplication // Bull. Amer. Math. Soc. 2008. V. 45. P. 247–284.
10. Brent R.P. Algorithms for matrix multiplications // Comput. Sci. Dept. Report CS 157. Stanford University, 1970.
11. Winograd S. On multiplication of 2×2 matrices // Linear Algebra and Appl. 1971. V. 4. P. 381–388.
12. Alekseyev V. B. On the complexity of some algorithms of matrix multiplication // J. Algorithms. 1985. V. 6. № 1. P. 71–85.
13. Laderman J. A noncommutative algorithm for multiplying 3×3 matrices using 23 multiplications // Bull Amer. Math. Soc. 1976. V. 82. № 1. P. 126–128.

14. *Johnson R.W., McLoughlin A.M.* Noncommutative bilinear algorithms for 3×3 matrix multiplication // *SIAM J. Comput.* 1986. V. 15. № 2. P. 595–603.
15. *Макаров О.М.* Некоммутативный алгоритм умножения квадратных матриц пятого порядка, использующий сто умножений // *Ж. вычисл. матем. и матем. физ.* 1987. Т. 27. № 2. С. 311–315.
16. *Bini D.* Relations between exact and approximate bilinear algorithms, applications // *Calcolo.* 1980. V. 17. P. 87–97.
17. *Бахвалов Н.С.* Численные методы. М.: Наука, 1975.
18. *Bini D., Capovani M., Lotti G., Romani F.* $O(n^{2.7799})$ complexity for approximate matrix multiplication // *Inform. Process. Lett.* 1979. V. 8. № 5. P. 234–235.
19. *Алексеев В.Б., Смирнов А.В.* О точной и приближенной билинейных сложностях умножения матриц размеров 4×2 и 2×2 // *Математика и информатика. 2. К 75-летию со дня рождения Анатолия Алексеевича Карацубы. Совр. пробл. матем.* 17, МИАН, М., 2013. С. 135–152.
20. *Hopcroft J.E., Kerr L.R.* On minimizing the number of multiplications necessary for matrix multiplication // *SIAM J. Appl. Math.* 1971. V. 20. № 1. P. 30–36.
21. *Bläser M.* On the complexity of the multiplication of matrices of small formats // *J. Complexity.* 2003. V. 19. Iss. 1. P. 43–60.
22. *Bini D., Lotti G.* Stability of fast algorithms for matrix multiplication // *Numer. Math.* 1980. V. 36. P. 63–72.